WAT.ai

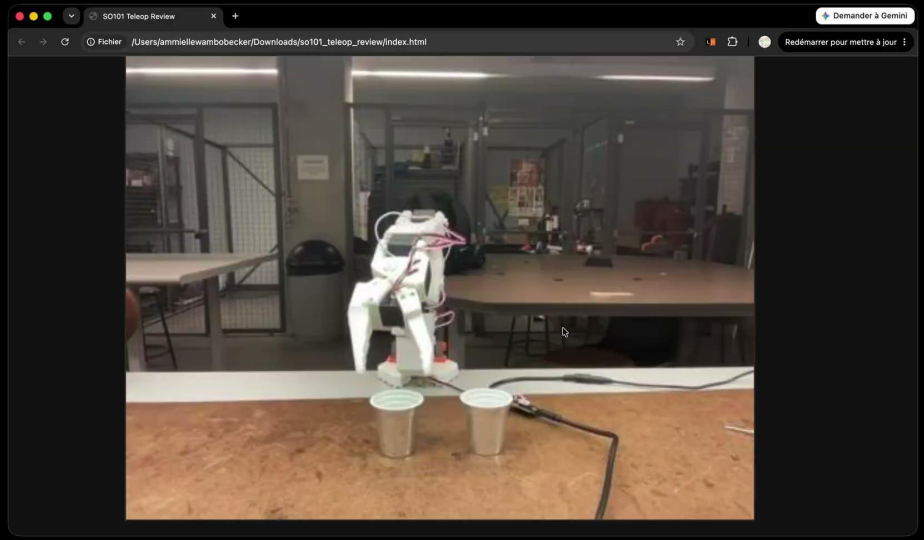
Robot Preference Learning

Teaching a robot arm to stir/pour liquid by asking which attempt looked better, instead of collecting more expert demonstrations, **the way you would coach a person.**

The Challenge

Robots need feedback to learn what “good” looks like

- Training a **reward model** for liquid pouring tasks means judging not just whether an episode succeeded, but **how well it succeeded.**
- That judgment has to come from somewhere, and the **source of feedback** shapes what the model actually learns.



"How do scripted rules, human labels, and vision-language model scoring compare as sources of preference feedback?"

WAT.ai

Why It Matters

Where “how well” matters more than “did it happen”

Warehouse & Fulfillment

Picking speed matters, but so does grip.

Agricultural Harvesting

Delicate crops can be picked yet damaged.

Assistive & Precision Robotics

Robotics operating around people need feedback on gentleness and care.

WAT.ai

How robots learn today, and why it is slow

30 min

of hand-driving

1 : 1

hour spent per hour gained

As good as its teacher, never better

The standard method is **imitation learning**: a person drives the robot through the task by hand, over and over, and the robot learns to copy the motion. It works, but every new task starts from zero. We drive ours with a second identical arm, a leader, and the robot follower mirrors it. That's **teleoperation**.

WAT.ai

Where this idea comes from

➡️ **arxiv.org/abs/1706.03741**

2017: researchers at OpenAI and DeepMind taught simulated robots backflips using only human A/B choices, no reward function written by hand

Why it worked: describing a good backflip in code is very hard; recognising one on sight is easy. Preferences move the work to the easy side

2022: the same recipe, RLHF, is what turned raw language models into ChatGPT and made them follow instructions

Our question: do preference labels pay off for physical manipulation, where every attempt costs real seconds on real hardware?

Three ways to say “A was better than B”

1/3

Scripted

1/3

Human

1/3

VLM

Idea: look at two attempts (A vs B) and pick the better one.
Scripted = a rule does it automatically from numbers the simulator reports.
Human = a person watches both clips and clicks.
VLM = an AI that can see, like Gemini, is shown the video and asked.
Everything downstream is held identical, so any difference in the trained robot comes from who did the judging.

WAT.ai

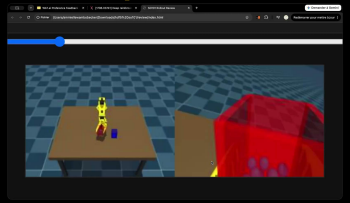
What we have collected

➡️ **huggingface.co/datasets/ammiellewb/wat.ai-so101**

Hardware

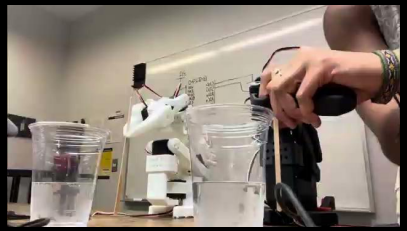
- 6 joints of action and state
- a front camera at 640x480

59 episodes



at 30 fps

51 episodes



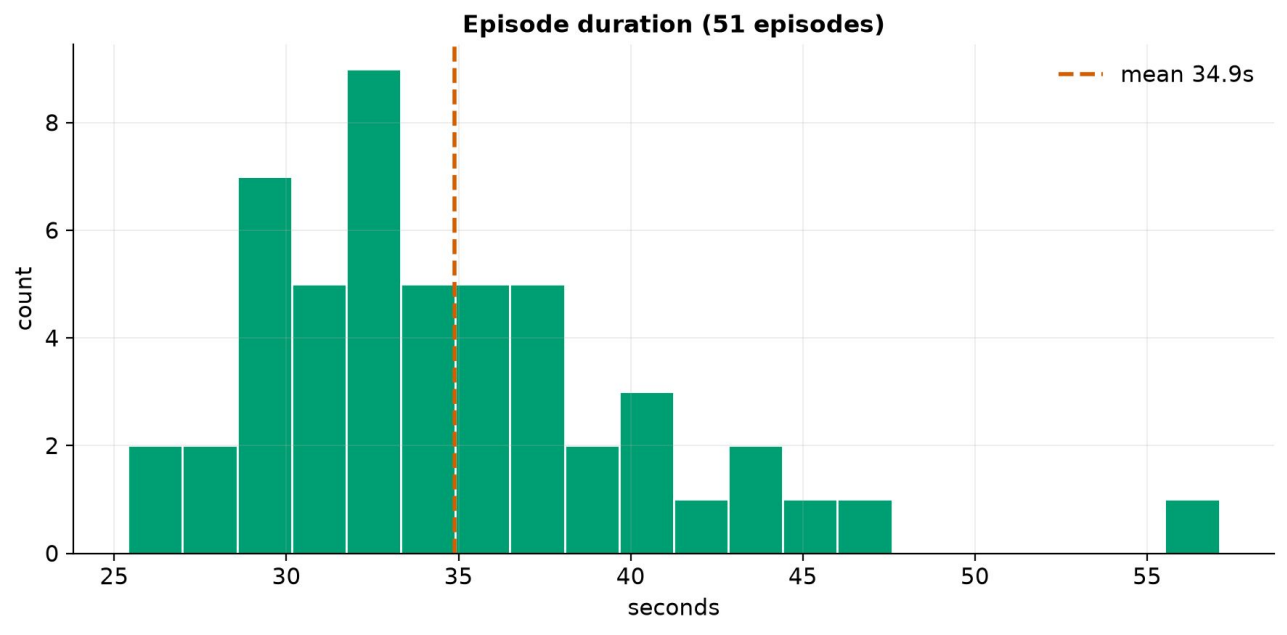
53,409 frames

Simulated

Pouring gives graded outcomes: how much landed, how much spilled, how smooth the tilt was.
Stirring's quality is ambiguous, which means human labellers agree with each other less.

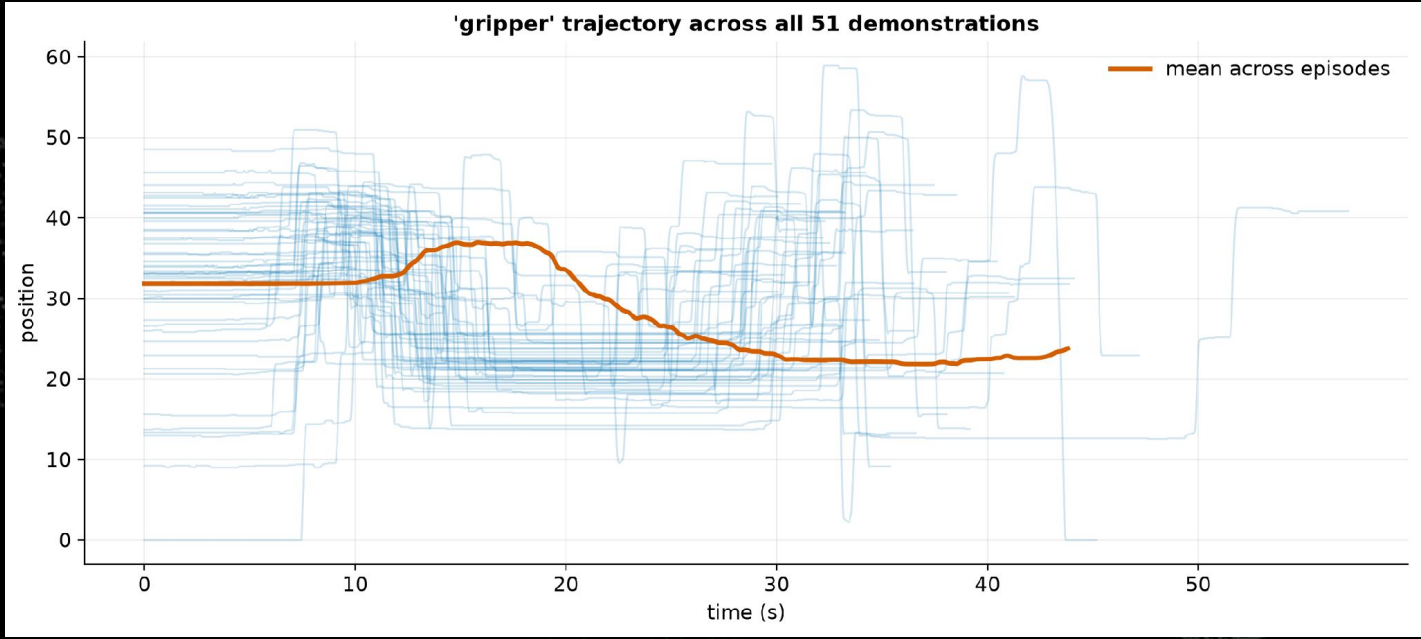
WAT.ai

Consistent length, not just consistent count



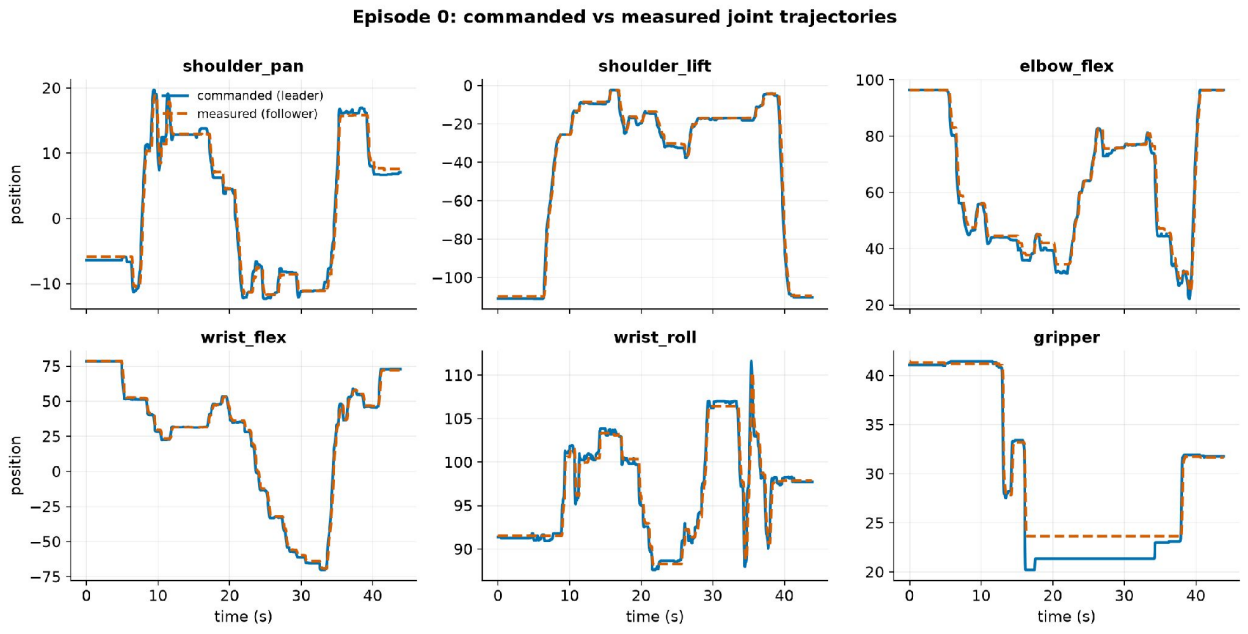
51 demonstrations, averaging 34.9 seconds, nearly all between 28 and 42. Tight timing is the first sign the operator ran the same strategy every time. That matters because the model we train, **Action Chunking with Transformers (ACT)**, learns purely by copying: it has no notion of a goal, only of what the human did. Inconsistent teaching produces a robot that hesitates at the moment it should commit.

Every demonstration has the same shape



The gripper opening across all 51 demonstrations, with the average in orange. Every run has the same shape: open, close on the cup around 10 to 13 seconds in, hold through the pour, release at the end. The exact timing wanders, the structure does not. That repeatability is what makes the data learnable.

The follower tracks the leader almost exactly



Blue is what the operator asked for, orange is what the robot actually did, across one full episode.
Five of the six joints track so closely the two lines are hard to tell apart, which means the hardware is faithfully recording our intent.
The gripper is the exception, and that gap is physics: the operator squeezes past where the jaw can close because the cup is in the way.

What a reward model is, and how we build one

The idea: a second model that watches an attempt and outputs a single score for how good it was

Training it: feed it pairs of attempts with the winner marked, and adjust until it scores winners above losers

Three of them: one per feedback source, so scripted, human and VLM each produce an independent scorer

Then compare: accuracy against held-out human labels, agreement between sources, and average score on successful pours

WAT.ai

Engineering Lessons

Consistency beats volume

One strategy, recorded the same way every time. Mixed styles average into hesitation.

Diversity must be injected

Preferences need good and bad attempts to compare. Clean data alone gives neither.

Infrastructure is the work

All three judges answer in the same format, so the comparison is fair.

Scripted labels stay in sim

Rules need simulator ground truth. On hardware only the AI judge runs without a person.

Ties mean no signal

If a judge calls most pairs equal, it has learned nothing.
Coarse metrics (yes/no) cause this.

Rollouts, not more demos

The spread comes from letting the trained robot try alone, with noise and moved cups.

VAT.ai

Next steps

Now: let the trained robot pour on its own, cup moved each time, until we have a mix of good and bad attempts to compare

Then: collect 150-300 "which one was better" answers from each of the three judges

Then: train a scorer from each set, retrain the robot on it, and measure whether the pours actually improve

August: publish the dataset format and failure analysis, CUCAI 2027

WAT.ai

Which pour was better?

Two clips, same task, same policy.
Pick one, then we will show you what the scripted rules and the VLM each said.

A negative result is still a result: VLM labels being too noisy to learn a reliable reward would be worth publishing, not a failure.

➡️
github.com/Reinforcement-Learning-RLHF

WAT.ai

VLM Scoring on MuJoCo Sim (42 pretrained ACT rollouts)



WAT.ai

VLM Scoring on SO-101 Robot Arm (51 clean expert demos)



WAT.ai